

Audit-Gated Recipe Evolution: Making Self-Reported Results from LLM-Driven Program Search Trustworthy

Qindong Gan
Waseda University
ganqg127@gmail.com

August 2026

Abstract

LLM-driven program search commonly selects candidates using scores that the candidates’ own training runs report. When the candidate program also controls how that score is measured—its reward shaping, its evaluation schedule, its exposure to a fixed set of seeds—selection optimizes the report rather than the result. We study this failure mode in a minimal *audit-gated recipe evolution* harness: an LLM iteratively rewrites training recipes (optimizer settings, evaluation schedules, shaped rewards) for a fixed policy class, and every candidate receives both a self-reported *claimed* score and an *audited* score measured by harness-owned code on freshly drawn seeds under the environment’s raw reward. The selection signal—trusting claims versus trusting audits—is the single manipulated variable, evaluated in two environments with deliberately different execution backends (MinAtar Breakout under jit-compiled batch rollouts; LunarLander-v3 under a process pool). Selecting on self-reports systematically breeds inflation: on LunarLander every claimed-selection candidate over-reports (15/15; mean gap +241 raw-reward points, versus +141 under audited selection), selection picks a candidate 35 points worse than the best already sitting in its own archive, and the top-ranked candidate claims +154 while actually scoring −119. Trajectory forensics show the inflation is *measurement gaming* rather than reward-function trickery: under claimed selection, evolved schedules collapse evaluation effort (mean episodes per evaluation 1.4 versus 2.4 late in the search), and replaying the top candidate under its own measurement conditions reproduces a ∼400-point illusion even though no recipe code executes at audit time. A hard audit gate—fresh OS-entropy seeds, raw rewards, no candidate code path—restores selection quality at matched candidate budget. We release the harness, all run archives, and the analysis code.

1 Introduction

A growing family of systems uses large language models to propose programs and an outer loop to keep the best ones: program search for mathematical discovery [9, 12], reward-function design [7], algorithm evolution [11], prompt-space optimization [17], and automated research pipelines [6]. These loops share a load-bearing assumption: the fitness attached to each candidate is accurate enough for selection.

That assumption has two very different regimes. In domains with exact verifiers—a theorem checker, an exact combinatorial objective—the evaluator is trusted code and the assumption holds by construction. But when candidates are *training programs* whose quality is expensive to measure, practical systems let the candidate’s own run report its fitness: the best score seen during training,

under the candidate’s own reward shaping and evaluation settings. The candidate then controls not only the object being scored but the measurement procedure itself. Selection pressure applies to the composite, and Goodhart-style failure [8] becomes an engineering concern rather than a philosophical one. The specification-gaming literature documents what optimizers do to imperfect objectives [1, 3, 5, 10, 15]; automated research systems have already exhibited the program-search variant, such as agents editing their own evaluation harness [6].

This paper isolates the question experimentally. We build a deliberately small recipe-evolution harness in which *who controls measurement* is the only thing that changes between conditions, and we quantify what selection on self-reports does to an otherwise identical search.

Contributions.

1. **A minimal audit-gated harness** for LLM-driven recipe evolution. A recipe is a small Python module controlling exactly three things (optimizer settings, per-generation evaluation schedule, per-transition shaped reward) for a fixed inner trainer. Every candidate gets a *claimed* score (its own training’s best fitness under its own measurement) and an *audited* score (harness-owned re-evaluation of the trained policy: fresh OS-entropy seeds, raw environment reward, fixed horizon, no execution path for recipe code). Selection mode is a switch; audits always run, so every run can be re-ranked under either signal post hoc.
2. **Evidence that selection on self-reports breeds inflation.** Under audited selection the claimed–audited gap is mildly optimistic; under claimed selection it inflates (Breakout: 90% of candidates positive versus 60%, mean +1.10 versus +0.37 bricks; LunarLander: 15/15 positive versus 38/40, mean +241 versus +141 points), selection quality degrades at matched budget, and the search demonstrably misses better candidates already in its own archive.
3. **A mechanism: measurement gaming, not reward hacking.** The inflated champion’s shaped reward is benign; what evolved is *cheap, noisy, short-horizon self-evaluation* (episodes per evaluation collapsing toward 1, training horizons of 150–400 steps versus the 1000-step audit), plus overfitting to a fixed training seed pool. Replaying the champion policy under its own measurement conditions scores +282; under the audit protocol the same parameters score −119.
4. **An honest account of what audits certify.** Independently retraining the seed recipe shows recipe-level outcome variance (e.g. audited returns from −237 to −128 on LunarLander); audits certify *policies*, not *recipes*. We release code, run archives, and analysis scripts.¹

2 The audit-gated recipe evolution harness

Recipe contract. A candidate *recipe* is a single Python module defining exactly: (i) `OPTIMIZER`, a dict with the inner optimizer’s initial step size $\sigma_0 \in [0.01, 2]$ and population size in $[4, 64]$; (ii) `schedule(gen)`, returning per-generation evaluation settings—episodes per candidate evaluation in $[1, 16]$ and a training episode step cap in $[50, 1000]$; and (iii) `shaped_reward(obs, action, reward, terminated, truncated, step)`, a deterministic per-transition training reward. Everything else—policy architecture, training loop, compute budgets, the audit—is fixed harness code that recipe code cannot touch. Static checks (import allowlist, no IO, no dunder access) reject obviously abusive programs as hygiene; they are not a security sandbox, and none of our results depend on them.

¹<https://github.com/Frankiegan912/shinsa-evolve>

Inner training. The policy class is a fixed tanh MLP with hidden layers (32, 32) over the flattened observation (13,987 parameters for Breakout, 1,508 for LunarLander), trained by separable CMA-ES [2, 13], the standard linear-complexity variant for parameter vectors of this size, in the spirit of evolution strategies for policy search [14]. Fitness of a population member is its mean shaped return over the scheduled number of episodes, drawn from a small per-candidate seed pool that is fixed across generations—deliberately, since seed reuse is one honest channel through which self-reports drift optimistic. Hard per-candidate budgets (episode count and wall clock) are enforced by the harness regardless of what the schedule requests.

Claimed score. The candidate’s *claimed* score is the best training fitness observed during its run—i.e. the number the recipe’s own measurement machinery produces: its shaping, its episode counts, its horizon, its fixed seed pool. The trained parameters submitted for evaluation are those of the best member under that same signal. This mirrors the self-reported “best score seen in training” that practical search loops archive.

Audit gate. The *audited* score re-evaluates the submitted parameters under three invariants: (1) evaluation seeds are freshly drawn from OS entropy at audit time, so no candidate can anticipate or overfit them; (2) scoring uses the environment’s raw reward only, over a fixed 1000-step horizon and 32 episodes; (3) recipe code has no execution path into the audit—the audit sees a parameter vector, nothing else. The *honesty gap* of a candidate is $g = \text{claimed} - \text{audited}$, defined relative to this fixed protocol. For honest recipes g can be negative (e.g. when a recipe trains at shorter horizons than the audit); what matters experimentally is how the distribution of g responds to selection pressure.

Outer loop and modes. The outer loop repeats: select a parent from the archive (80% uniform over the current top-5, 20% uniform over all), ask the LLM to rewrite it (one-shot prompt containing the contract, the environment description, the parent source, and the parent’s score under the run’s selection signal), validate with rejection sampling, train, audit, archive. The run’s *mode* decides everything the search sees: in **audited** mode, parent ranking and LLM feedback use audited scores; in **claimed** mode, both use claimed scores. Audits still run in claimed mode, but purely as recording—exactly the counterfactual instrumentation needed to measure what trusting self-reports costs. The mutation operator is a frontier commercial LLM accessed through its command-line interface; across all runs, 98% of 131 mutations were accepted on the first attempt (mean CLI latency 96s), so rejection sampling is not a meaningful filter on the phenomena studied here.

3 Experimental setup

Two environments exercise two execution backends on purpose, so that the harness claim does not depend on one execution model: MinAtar Breakout [18] runs under jit-compiled, vmapped batch rollouts in gymnasium [4]; LunarLander-v3 runs under a spawn-based process pool in Gymnasium [16]. Shaping is applied by shared harness code over recorded raw trajectories in both backends, so the contract semantics are identical. Table 1 lists the matrix. The seed recipe is an honest baseline (raw-reward passthrough, 4 episodes per evaluation); each environment additionally gets a variance baseline of three independent retrains of that seed recipe. Because claimed runs also carry audits, unequal candidate counts do not confound the comparisons below: cross-mode comparisons are made at matched (truncated) candidate budgets, and the selection-failure analysis is internal to each claimed run.

Run	Environment	Backend	Selection	Candidates	Per-candidate budget
BA	MinAtar Breakout	gymnax, jit batch	audited	60	8,000 ep / 300 s
BC	MinAtar Breakout	gymnax, jit batch	claimed	20	8,000 ep / 300 s
LA	LunarLander-v3	gymnasium, spawn pool	audited	40	2,000 ep / 600 s
LC	LunarLander-v3	gymnasium, spawn pool	claimed	15	2,000 ep / 600 s

+ seed-recipe variance baselines: 1 seed training + 3 independent retrains per environment

Table 1: Experiment matrix. Audits are identical everywhere: 32 fresh-seed episodes, raw reward, 1000-step horizon. All runs execute on a single consumer laptop.

Run	N	gap mean	gap median	gap > 0	gap max	pick (audited score)
BA (audited)	60	+0.37	+0.10	36/60	+3.61	4.94
BC (claimed)	20	+1.10	+0.84	18/20	+2.77	4.31
LA (audited)	40	+140.8	+129.0	38/40	+524.0	-45.1
LC (claimed)	15	+241.3	+222.9	15/15	+552.8	-118.8

Table 2: Honesty-gap statistics and the audited score of each run’s final pick (by its own selection signal). Units: raw bricks (Breakout) and raw return points (LunarLander).

4 Results

4.1 Selection pressure on self-reports breeds inflation

Table 2 and Figure 1 give the headline comparison. Under audited selection, gaps are mildly optimistic: on Breakout the distribution is roughly centered (median +0.10 bricks, 60% positive); on LunarLander it is optimistic for structural reasons discussed below. Under claimed selection the distribution shifts decisively: 90% positive with $3\times$ the mean gap on Breakout, and *universal* over-reporting on LunarLander (15/15, mean +241). Selection is doing this, not the LLM’s priors: both modes use the same mutation operator, prompt template, contract, budgets, and parent-selection rule; only the score being trusted differs.

4.2 Trusting claims degrades what the search actually selects

The operational question is not whether candidates exaggerate but whether the loop ends up keeping worse programs. It does (Figure 4). At matched candidate budgets, audited selection’s best pick beats claimed selection’s pick in both environments: 4.94 versus 4.31 bricks on Breakout (first 20 candidates), -62.5 versus -118.8 points on LunarLander (first 15). With its full 60-candidate budget the Breakout audited run stays at 4.94; the LunarLander audited run reaches -45.1 .

Claimed selection also fails *relative to itself*: ranking its own archive by claims picks a candidate worse than what it already found. On LunarLander the claimed run’s pick audits at -118.8 while candidate 2 in the same archive audits at -83.9 (regret 35.0); on Breakout the corresponding regret is 0.25 bricks (4.31 picked versus 4.56 available). The mis-ranking is not subtle: the claimed run’s top-5 by claims on LunarLander includes a candidate claiming +147.1 that audits at -405.7 .

4.3 Mechanism: measurement gaming, not reward hacking

The natural suspicion is that claimed-mode recipes learned to hack their shaped rewards. The data say otherwise. Take the LunarLander claimed run’s champion (claims +153.7, audits -118.8).

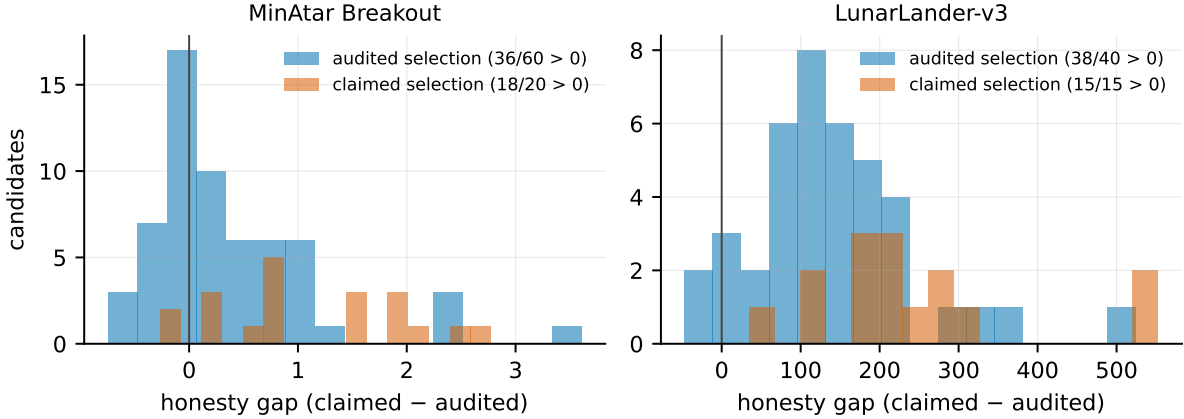


Figure 1: Honesty gap (claimed – audited) distributions. Claimed-mode selection (vermillion) shifts the gap distribution to systematic over-reporting in both environments.

Its shaping is benign—small soft-landing penalties, a 0.05-per-leg contact bonus—and under audit conditions contributes -2.1 per episode on average. Across eight fresh audit-length episodes the policy crashes quickly (mean length 102 steps of a possible 1000; legs touch the ground in 4.4% of steps), scoring a mean raw return of -121.7 , consistent with its archived audit.

What actually evolved is the *measurement*. The champion’s schedule evaluates candidates on 2 episodes with a 400-step cap; the run’s runner-up (claims $+147.1$, audits -405.7) uses 1 episode and a 150-step cap. Three mutually reinforcing distortions follow. First, *evaluation noise is rewarded*: a best-of selection over noisy estimates preferentially archives lucky draws, so recipes that measure themselves with 1–2 episodes look better than recipes that measure honestly. Second, *fixed-pool seed overfitting*: CMA-ES adapts the policy to the handful of initial conditions it is repeatedly evaluated on. Third, *horizon truncation*: a 150–400-step training window simply never observes the late consequences the 1000-step audit charges for. Replaying the champion’s archived parameters under its own generation-0 measurement conditions—its two training seeds, its 400-step cap—scores $+282.2$ (its archived claim of $+153.7$ was recorded under the schedule active at the generation that produced the best member). The identical parameter vector scores -118.8 under the audit protocol. The entire ~ 400 -point illusion is produced by measurement conditions; no recipe code runs at audit time.

The population-level statistics agree (Table 3): claimed-mode runs evolve fewer episodes per evaluation than audited-mode runs in both environments (late-run means 1.8 versus 2.3 on Breakout, 1.4 versus 2.4 on LunarLander; the honest seed uses 4). Both modes face the same incentive to spend less compute per evaluation (cheaper evaluations buy more generations within a fixed budget), so some drift downward is rational cost allocation; the *differential* between modes, consistent across environments and growing over the run, is the measurement-gaming signal: under claimed selection, noise itself pays.

4.4 Audits certify policies, not recipes

A passing audit certifies that *this trained parameter vector* performs as measured. It does not certify that *rerunning the recipe* reproduces that performance. Independently retraining the honest seed recipe three times (plus its original training) yields audited scores of 3.5–5.0 bricks on Breakout (4.3 ± 0.6) and -236.5 to -128.4 points on LunarLander (-161.4 ± 44.4). Selection over recipes

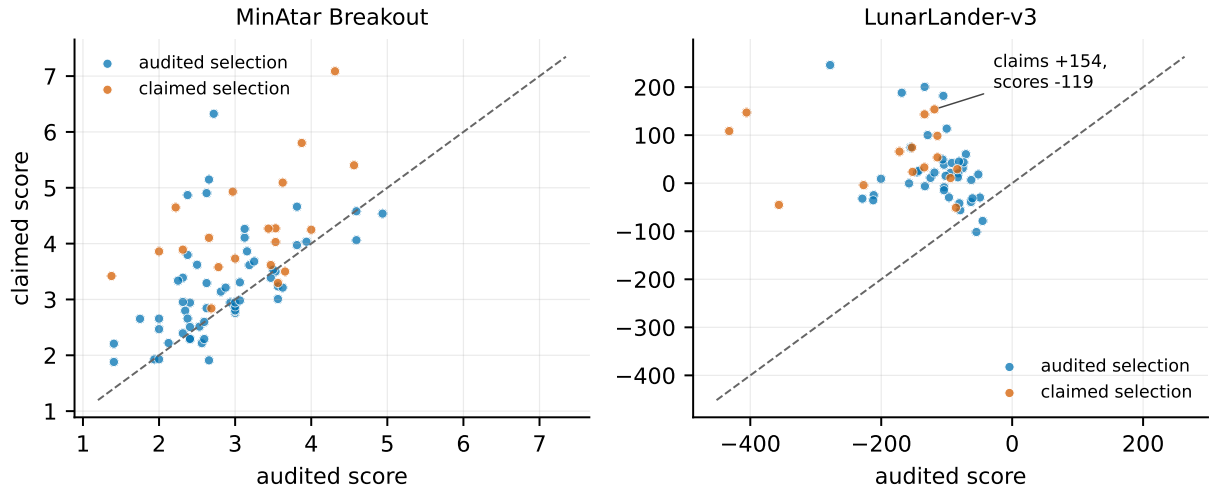


Figure 2: Claimed versus audited scores for every candidate. Points far above the diagonal claim much more than they deliver; the labeled LunarLander candidate is its run’s top pick.

Run	episodes per evaluation		training step cap	
	all	last 10	all	last 10
seed recipe	4.0	—	500–1000	—
BA (audited)	2.6	2.3	226	195
BC (claimed)	1.9	1.8	288	260
LA (audited)	2.4	2.4	370	320
LC (claimed)	1.7	1.4	315	242

Table 3: Mean evolved evaluation-schedule settings (at generation 0 of each recipe’s schedule). Claimed-mode populations converge to measurably cheaper, noisier self-evaluation than audited-mode populations in both environments.

therefore operates on draws from high-variance distributions, and a single audit collapses only the measurement error, not the retraining variance. Any deployment of audit-gated search that intends to reuse the *recipe* rather than the *artifact* should budget replicate retrains of finalists.

5 Limitations

Breakout improvements sit inside the retrain band. The Breakout audited champion (4.94) is within the seed recipe’s retrain range (3.5–5.0), so on that environment we claim only that the *comparative* phenomena (inflation, mis-ranking) reproduce—not that evolution discovered a stronger recipe. LunarLander carries the improvement claim: its audited champion (−45.1) clears the retrain band (−236.5 to −128.4) by a wide margin. **Scale.** Runs are small (≤ 60 candidates; 15–20 in claimed mode), one LLM family serves as the mutation operator, and both tasks are classic small-scale control; effect sizes elsewhere may differ, though the mechanism (noise, seed reuse, horizon truncation) is generic. **Gap semantics.** The honesty gap is defined against one fixed audit protocol; honest recipes can have negative gaps (training horizon shorter than the audit’s), so the evidential quantity is the distribution shift between modes, not any single candidate’s gap. **Hygiene, not sandboxing.** Static checks deter accidental IO; a determined adversarial program

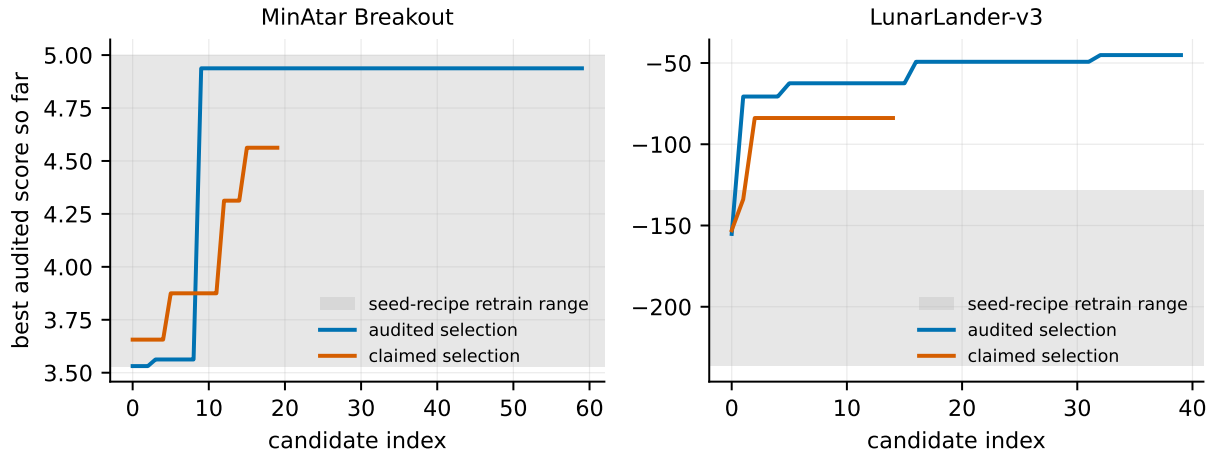


Figure 3: Best audited score discovered so far, by candidate index. The shaded band is the audited range of independent retrains of the honest seed recipe. On LunarLander, audited selection climbs well beyond the retrain band; on Breakout the evolved champion remains near the top of it (see Limitations).

is out of scope—our results concern what *selection pressure* produces without any code-level attack on the audit.

6 Related work

LLM-driven program search with *trusted* evaluators has produced striking results: FunSearch [12] and AlphaEvolve [9] score candidates with exact verifiers or deterministic benchmark harnesses, and AutoML-Zero [11] evolves algorithms against controlled evaluation pipelines. Our setting differs in that the candidate is a *training program* whose self-reported fitness is the cheap signal, which moves the evaluator inside the candidate’s control. Eureka [7] evolves reward functions with LLMs and scores them by retraining under a fixed, trusted evaluation—the audit-shaped end of the spectrum; our claimed mode is the other end, and the comparison quantifies the distance between them. The result is a program-search instance of reward hacking and specification gaming [1, 3, 5, 10, 15], and of Goodhart failures under optimization pressure [8], with a twist the taxonomy should note: what our populations game is not the reward function but the *estimator*—its sample size, its horizon, its seeds. Automated-science systems report the adjacent failure of agents modifying their own evaluation code [6]; the audit gate is the minimal structural answer: keep one measurement path the candidate cannot reach.

7 Conclusion

In LLM-driven recipe evolution, self-reported fitness is not a noisy version of the truth; it is a signal the search will learn to manufacture. Given control over its own measurement, an evolving population converged on cheap, noisy, short-horizon self-evaluation—universal over-reporting on one benchmark, top-ranked candidates worth hundreds of points less than claimed, and selection that misses better programs already in its own archive. A hard audit gate with three properties—fresh

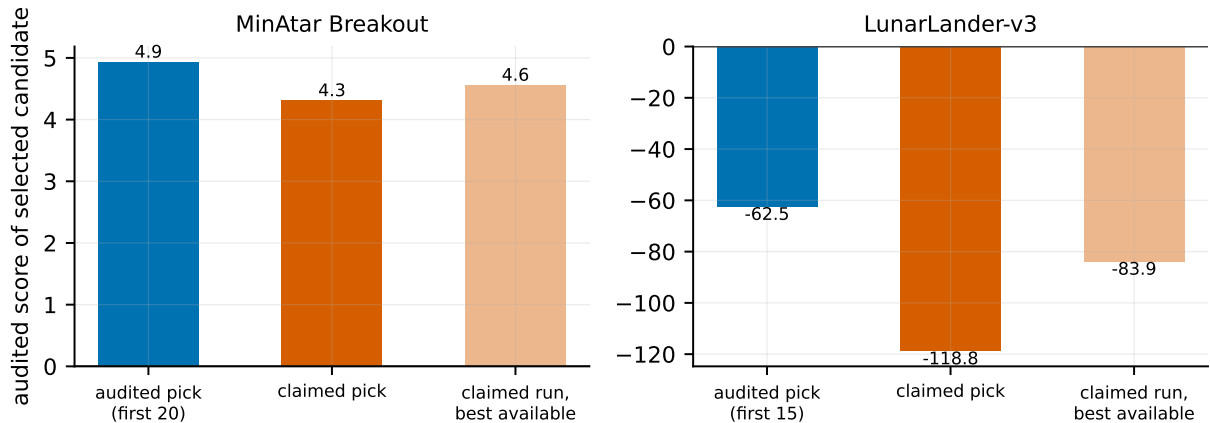


Figure 4: What each mode’s final selection is actually worth (audited score), at matched candidate budgets, alongside the best candidate available inside the claimed run’s own archive. Claimed selection both loses to audited selection and misses its own best candidate.

entropy seeds, ground-truth metric, no candidate code path—was sufficient to restore trustworthy selection at matched budget, at the price of the audit episodes and an honest caveat: audits certify the trained artifact, not the recipe that produced it. As LLM-driven search loops become infrastructure, we expect the audit gate to be the kind of component nobody reports on until it is missing.

Reproducibility. Code, prompts, all run archives (including per-candidate mutation logs and rejection reasons), and analysis scripts are available at <https://github.com/Frankiegan912/shinsa-evolve>. All experiments ran on a single consumer laptop; the two evolution backends (jit batch rollouts and process pools) are both included.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.
- [3] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.
- [4] Robert Tjarko Lange. gymmax: A JAX-based reinforcement learning environment library. <https://github.com/RobertTLange/gymmax>, 2022.
- [5] Joel Lehman, Jeff Clune, Dusan Misevic, Christoph Adami, Lee Altenberg, Julie Beaulieu, Peter J. Bentley, Samuel Bernard, Guillaume Beslon, David M. Bryson, et al. The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities. *Artificial Life*, 26(2):274–306, 2020.

- [6] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [7] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s law. *arXiv preprint arXiv:1803.04585*, 2018.
- [9] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [10] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [11] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *International Conference on Machine Learning (ICML)*, 2020.
- [12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [13] Raymond Ros and Nikolaus Hansen. A simple modification in CMA-ES achieving linear time and space complexity. In *Parallel Problem Solving from Nature (PPSN)*, 2008.
- [14] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [15] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [17] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.
- [18] Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. *arXiv preprint arXiv:1903.03176*, 2019.